

Avenida

Nome do arquivo: `avenida.c`, `avenida.cpp`, `avenida.java`, `avenida.js` ou `avenida.py`

Luiza está se preparando para começar a estudar em uma nova escola que será inaugurada na avenida em que ela mora. A avenida possui 2,000 metros de comprimento e existe um ponto de ônibus a cada 400 metros, incluindo no início e no fim da avenida. A tabela abaixo indica a distância de cada ponto de ônibus para o início da avenida.

Ponto #1	Ponto #2	Ponto #3	Ponto #4	Ponto #5	Ponto #6
0 m	400 m	800 m	1200 m	1600 m	2000 m

A casa de Luiza está localizada no início da avenida, junto ao primeiro ponto de ônibus. A escola, por outro lado, está localizada a uma distância D do início da avenida.

Luiza pretende pegar o ônibus na porta de casa, descer no ponto de ônibus mais próximo da escola e andar a pé o restante do trajeto. Assim, por exemplo, se a escola está a uma distância $D = 720$ m do início da avenida, ela vai descer no terceiro ponto de ônibus, localizado a 800 metros do início, e andar 80 metros (em direção ao início da avenida) para chegar à escola.

Luiza pediu sua ajuda para descobrir quantos metros ela precisará andar: dada a distância em metros D da escola para o início da avenida, determine qual a distância entre a escola e o ponto de ônibus mais próximo.

Entrada

A entrada é composta por uma única linha contendo um único inteiro D , representando a distância em metros da escola para o início da avenida.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, a distância mínima em metros que Luiza precisará andar entre um ponto de ônibus e a escola.

Restrições

- $0 \leq D \leq 2000$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (30 pontos):** $D < 800$.
- **Subtarefa 3 (70 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 720	Exemplo de saída 1 80
Exemplo de entrada 2 30	Exemplo de saída 2 30
Exemplo de entrada 3 1434	Exemplo de saída 3 166
Exemplo de entrada 4 400	Exemplo de saída 4 0

Alfabeto alienígena

Nome do arquivo: `alfabeto.c`, `alfabeto.cpp`, `alfabeto.java`, `alfabeto.js` ou `alfabeto.py`

Mais uma vez, o OBI (Órgão Brasileiro de Inteligência) está preocupado com a possibilidade da existência de vida alienígena. Os diretores do órgão suspeitam que os alienígenas existem, conseguiram se infiltrar dentro da instituição e tem se comunicado secretamente. Os agentes do OBI se comunicam usando o dispositivo de mensagens oficial do órgão, que possui as seguintes teclas: letras maiúsculas de A a Z, letras minúsculas de a a z, dígitos de 0 a 9, operadores aritméticos (+, -, *, /), *hashtag* (#) e ponto de exclamação (!).

O OBI descobriu que, sempre que dois alienígenas se comunicam entre si usando o dispositivo, eles usam um alfabeto alienígena que possui um conjunto específico de símbolos. Assim, uma mensagem pode ter sido escrita por alienígenas se, e somente se, todos os símbolos que compõem ela pertencem ao alfabeto alienígena. Por exemplo, se o alfabeto alienígena for composto pelas caracteres `!`, `1`, `o` e `b`, a mensagem `ob1!!` é uma mensagem que poderia ser escrita por alienígenas. Por outro lado, a mensagem `Obi!` não poderia ter sido escrita por alienígenas pois tanto o primeiro caractere `O` (maiúsculo) quanto o terceiro caractere `i` não fazem parte do alfabeto alienígena.

Você foi contratado para ajudar o OBI a identificar os invasores: dadas a lista de caracteres usados no alfabeto alienígena e uma mensagem enviada pelo dispositivo, determine se a mensagem poderia ter sido escrita por alienígenas ou não.

Entrada

A primeira linha de entrada contém dois inteiros K e N separados por um espaço em branco, indicando, respectivamente, o número de caracteres presentes no alfabeto alienígena e o número de caracteres da mensagem enviada.

A segunda linha de entrada contém K caracteres distintos representando os caracteres pertencentes ao alfabeto alienígena.

A terceira linha de entrada contém N caracteres (não necessariamente distintos) representando a mensagem enviada.

Saída

Seu programa deverá imprimir uma única linha contendo um único caractere: se a mensagem pode ter sido escrita no alfabeto alienígena, imprima a letra 'S' maiúscula; caso contrário, imprima a letra 'N' maiúscula.

Restrições

- $1 \leq K \leq 68$
- $1 \leq N \leq 1000$
- Todos os caracteres usados no alfabeto ou na mensagem pertencem à lista a seguir:

`abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+ -*/#!`

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (33 pontos):** $K = 1$, ou seja, o alfabeto alienígena possui apenas um símbolo (*veja o exemplo 2*).
- **Subtarefa 3 (29 pontos):** $K = 26$ e o alfabeto alienígena é exatamente o nosso alfabeto de letras minúsculas, ou seja, `abcdefghijklmnopqrstuvwxyz` (*veja o exemplo 3*).
- **Subtarefa 4 (38 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 4 5 !1ob ob1!!	Exemplo de saída 1 S
Exemplo de entrada 2 1 5 a aabab	Exemplo de saída 2 N
Exemplo de entrada 3 26 32 abcdefghijklmnopqrstuvwxyz olimpiadabrasileiradeinformatica	Exemplo de saída 3 S
Exemplo de entrada 4 11 7 0123+-!ABCD OBI!OBI	Exemplo de saída 4 N

Atletismo

Nome do arquivo: `atletismo.c`, `atletismo.cpp`, `atletismo.java`, `atletismo.js` ou `atletismo.py`

Os Jogos Olímpicos de Paris 2024 acabaram de terminar e mais uma vez reviveram na população o interesse por esportes. Uma das modalidades que mais receberam atenção dos espectadores foi o atletismo, no qual a corrida de 100 metros é uma das principais provas.

O sucesso foi tanto que o estádio onde Tiago trabalha recebeu muitas inscrições para as aulas de atletismo. Tiago está empolgado com o novo grupo, mas teme que a grande quantidade de alunos torne difícil determinar os resultados das corridas.

Em uma corrida com N atletas, os atletas são numerados de 1 a N e o sistema automatizado do estádio é capaz de registrar a ordem na qual os atletas passaram pela linha de chegada. Vale ressaltar que este sistema é muito preciso, e portanto nunca há empate entre dois atletas. Tiago gostaria de usar essas informações para descobrir em qual posição cada atleta ficou no *ranking* da corrida.

Por exemplo, se $N = 6$ e a ordem em que os atletas cruzaram a chegada foi, do primeiro ao último,

5, 2, 4, 6, 3, 1

então o atleta com número 1 ficou na posição 6, o atleta com número 2 ficou na posição 2, o atleta com número 3 ficou na posição 5 e assim em diante. A tabela abaixo indica a posição de cada atleta.

Número do atleta	1	2	3	4	5	6
Posição no <i>ranking</i>	6	2	5	3	1	4

Dadas a quantidade N de atletas em uma corrida e a ordem em que os N atletas cruzaram a linha de chegada, sua tarefa é escrever um programa que determine, para cada um dos N atletas, a posição dele no *ranking* da corrida.

Entrada

A primeira linha de entrada contém um único inteiro N representando a quantidade de atletas que participaram da corrida.

As próximas N linhas contém cada uma um inteiro e representam a ordem em que os atletas cruzaram a linha de chegada, do primeiro ao último. Ou seja, a i -ésima dessas linhas contém o número do i -ésimo atleta a cruzar a linha de chegada.

Saída

Seu programa deverá imprimir N linhas, cada uma contendo um único inteiro. A i -ésima linha deverá conter a posição no *ranking* do atleta com número i .

Restrições

- $1 \leq N \leq 100\,000$
- Cada inteiro de 1 a N aparece exatamente uma vez na ordem de chegada

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (40 pontos):** $N \leq 1000$.
- **Subtarefa 3 (60 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 6 5 2 4 6 3 1	Exemplo de saída 1 6 2 5 3 1 4
Exemplo de entrada 2 3 2 3 1	Exemplo de saída 2 3 1 2
Exemplo de entrada 3 9 9 5 1 3 6 4 2 7 8	Exemplo de saída 3 3 7 4 6 2 5 8 9 1

Dança de Formatura

Nome do arquivo: `danca.c`, `danca.cpp`, `danca.java`, `danca.js` ou `danca.py`

A escola de educação básica do seu bairro está organizando uma festa de formatura para os graduandos deste ano. Para isso, eles pediram que a OBI (Organização de Brincadeiras Infantis) desenvolva uma dança que os alunos possam apresentar aos pais durante a formatura.

A dança da OBI é dançada em uma pista quadriculada com N linhas e M colunas, sempre com exatamente um aluno em cada quadrado da pista. Os alunos são numerados de 1 a $N \times M$ de acordo com a sua posição inicial na pista em ordem crescente de linha e coluna, nesta ordem, a partir do quadrado $(1, 1)$. O exemplo abaixo, para $N = 4$ e $M = 3$, indica o número do aluno em cada quadrado da pista no início da dança; o aluno de número 7, por exemplo, inicia no quadrado $(3, 1)$.

	Col. 1	Col. 2	Col. 3
Linha 1	1	2	3
Linha 2	4	5	6
Linha 3	7	8	9
Linha 4	10	11	12

A cada passo da dança, o professor dá aos alunos uma das duas ordens abaixo:

- “L a b ” (onde a e b são inteiros distintos), ordenando que os alunos da a -ésima linha troquem de linha com os alunos da b -ésima linha, mantendo a coluna de cada um – ou seja, o aluno na célula $(a, 1)$ troca com o aluno na célula $(b, 1)$, $(a, 2)$ troca com $(b, 2)$ e assim por diante.
- “C a b ” (onde a e b são inteiros distintos), ordenando que os alunos da a -ésima coluna troquem de coluna com os alunos da b -ésima coluna, mantendo a linha de cada um – ou seja, o aluno na célula $(1, a)$ troca com o aluno na célula $(1, b)$, $(2, a)$ troca com $(2, b)$ e assim por diante.

A figura abaixo ilustra o progresso da dança para $N = 4$ e $M = 3$ com os três primeiros passos sendo “C 1 3”, “L 1 4” e “C 3 2”, nesta ordem.

1	2	3		3	2	1		12	11	10		12	10	11
4	5	6	$\xrightarrow{\text{C } 1 \ 3}$	6	5	4		6	5	4	$\xrightarrow{\text{L } 1 \ 4}$	6	4	5
7	8	9		9	8	7		9	8	7		9	7	8
10	11	12		12	11	10	$\xrightarrow{\text{C } 3 \ 2}$	3	2	1		3	1	2

A escola gostou muito da dança inventada pela OBI e deseja usá-la na formatura. Porém, os pais não querem perder seus filhos de vista e pediram sua ajuda para saber quais serão as posições de seus filhos ao término da dança.

Sua tarefa é: dadas as dimensões N e M da pista de dança, a quantidade P de passos da dança e a ordem dada pelo professor a cada passo, determine qual aluno estará em cada quadrado da pista ao fim da dança.

Entrada

A primeira linha da entrada é composta por três inteiros N , M e P indicando, respectivamente, o número de linhas da pista de dança, o número de colunas da pista de dança, e o número de passos da dança.

As próximas P linhas descrevem as ordens dadas pelo professor. A i -ésima dessas linhas contém uma letra **maiúscula** O_i , que pode ser ‘L’ ou ‘C’, seguida de dois inteiros distintos A_i e B_i .

- Se $O_i = \text{‘L’}$, o professor ordenou a troca das linhas A_i e B_i .
- Se $O_i = \text{‘C’}$, o professor ordenou a troca das colunas A_i e B_i .

Saída

Seu programa deverá imprimir N linhas, cada uma contendo M inteiros. O j -ésimo inteiro da i -ésima linha deve ser o número do aluno que terminará a dança na i -ésima linha e j -ésima coluna da pista.

Restrições

- $1 \leq N \leq 1\,000\,000$
- $1 \leq M \leq 1\,000\,000$
- $1 < N \times M \leq 1\,000\,000$
- $1 \leq P \leq 500\,000$
- $O_i = \text{‘L’}$ ou $O_i = \text{‘C’}$
- Se $O_i = \text{‘L’}$, $1 \leq A_i, B_i \leq N$
- Se $O_i = \text{‘C’}$, $1 \leq A_i, B_i \leq M$
- $A_i \neq B_i$

Atenção: Observe que não é possível declarar $1\,000\,000 \times 1\,000\,000$ inteiros (com matriz, vetor etc.) sem estourar o limite de memória (isto causaria erros no programa pois tentaria usar milhares de GB de memória). Preste atenção ao limite $N \times M \leq 1\,000\,000$, que garante que a pista de dança sempre terá no máximo 1 000 000 alunos.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (20 pontos):**
 - $N = 1$
 - $M \leq 1000$
 - $P \leq 1000$
- **Subtarefa 3 (20 pontos):**
 - $N \leq 1000$
 - $M \leq 1000$
 - $P \leq 1000$

- **Subtarefa 4 (31 pontos):**

– $M = 2$ (Veja o exemplo 3.)

- **Subtarefa 5 (29 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 4 3 3 C 1 3 L 1 4 C 3 2	Exemplo de saída 1 12 10 11 6 4 5 9 7 8 3 1 2
Exemplo de entrada 2 1 6 4 C 2 5 C 1 2 C 4 3 C 1 2	Exemplo de saída 2 1 5 4 3 2 6
Exemplo de entrada 3 5 2 6 C 1 2 L 1 3 L 1 4 C 2 1 L 5 3 C 2 1	Exemplo de saída 3 8 7 4 3 10 9 6 5 2 1

Concatena Dígitos

Nome do arquivo: `concatena.c`, `concatena.cpp`, `concatena.java`, `concatena.js` ou `concatena.py`

Beatriz está se divertindo com o novo jogo que ela inventou, o *Concatena Dígitos*! Concatenar é o nome que ela dá ao processo de pegar dois dígitos e juntá-los de modo a criar um número de dois dígitos. Por exemplo, ao concatenar os dígitos 2 e 9, nessa ordem, Beatriz cria o número 29.

Beatriz gosta de trabalhar com muitos dígitos. Por isso, ela utiliza uma lista com N dígitos de 1 a 9 (observe que ela **não** usa o dígito 0) com posições numeradas de 1 a N (da esquerda para a direita) para escolher qual par ela irá concatenar. O exemplo abaixo ilustra uma lista com $N = 3$.

1 1 2

Para concatenar dígitos, Beatriz primeiro escolhe uma posição na lista, depois escolhe outra posição **diferente da primeira**, e concatena, nesta ordem, os dígitos que estão nas posições escolhidas (ou seja, o dígito na primeira posição escolhida se torna o dígito das dezenas e o dígito na segunda posição escolhida se torna o dígito das unidades). Por exemplo, na lista acima, uma concatenação possível é escolher a primeira posição, que possui o dígito 1, então escolher a terceira posição, que possui o dígito 2, e juntá-las para gerar o número 12. No total, existem 6 concatenações possíveis:

- 1 (primeira posição) e 1 (segunda posição) $\rightarrow$ 11
- 1 (primeira posição) e 2 (terceira posição) $\rightarrow$ 12
- 1 (segunda posição) e 1 (primeira posição) $\rightarrow$ 11
- 1 (segunda posição) e 2 (terceira posição) $\rightarrow$ 12
- 2 (terceira posição) e 1 (primeira posição) $\rightarrow$ 21
- 2 (terceira posição) e 1 (segunda posição) $\rightarrow$ 21

Chamamos de *potencial* de uma lista de dígitos a soma de todas as concatenações possíveis. Por exemplo, o potencial da lista descrita acima é

$$11 + 12 + 11 + 12 + 21 + 21 = 88.$$

Similarmente, podemos calcular que a lista com os dígitos 1 1 2 3 9 possui potencial 704.

Também definimos o *potencial de um intervalo contíguo* da lista de dígitos como o potencial da lista obtida ao considerar apenas esse intervalo. Por exemplo, ao considerar somente o intervalo $[1, 3]$ (as três primeiras posições) da lista 1 1 2 3 9, obtemos a lista 1 1 2, e portanto o intervalo $[1, 3]$ da lista 1 1 2 3 9 possui potencial 88 (como vimos antes).

Beatriz acabou de criar uma nova lista de dígitos e pretende escolher um intervalo contíguo para brincar. Para isso, ela gostaria de saber o potencial de diversos intervalos contíguos da lista. Mais especificamente, Beatriz vai te fazer Q perguntas no seguinte formato: dado um intervalo contíguo $[L, R]$ da lista de dígitos, qual o potencial do intervalo $[L, R]$?

Entrada

A primeira linha da entrada contém dois números inteiros, N e Q , o número de dígitos da lista de Beatriz e a quantidade de perguntas que ela vai fazer.

A segunda linha da entrada contém N dígitos D_i entre 1 e 9 representando a lista de Beatriz.

As próximas Q linhas contém as perguntas de Beatriz. A i -ésima destas linhas contém dois inteiros L_i e R_i , indicando que Beatriz quer saber o potencial do intervalo $[L_i, R_i]$ da lista.

Saída

Seu programa deverá produzir Q linhas. A i -ésima dessas linhas deve conter um único inteiro, o potencial do intervalo entre L_i e R_i , inclusive.

Restrições

- $1 \leq N \leq 100\,000$
- $1 \leq Q \leq 100\,000$
- $1 \leq D_i \leq 9$ para todo $1 \leq i \leq N$
- $1 \leq L_i \leq R_i \leq N$ para todo $1 \leq i \leq Q$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (40 pontos):**
 - $N \leq 300$
 - $Q \leq 300$
- **Subtarefa 3 (28 pontos):**
 - $N \leq 4000$
 - $Q \leq 4000$
- **Subtarefa 4 (32 pontos):** Sem restrições adicionais.

Exemplos

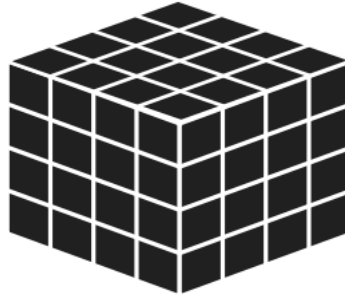
Exemplo de entrada 1	Exemplo de saída 1
5 4	88
1 1 2 3 9	704
1 3	132
1 5	0
2 4	
3 3	

Cubo Preto

Nome do arquivo: `cubo.c`, `cubo.cpp`, `cubo.java`, `cubo.js` ou `cubo.py`

Ana comprou um cubo de madeira de lado N cm (ou seja, dimensões $N \times N \times N$ centímetros) e o pintou todo de preto. Depois disso, ela cortou o cubo em N^3 cubinhos de lado 1 cm (ou seja, dimensões $1 \times 1 \times 1$ centímetro). Após o corte, alguns cubinhos terão nenhuma face pintada de preto, alguns terão exatamente uma face pintada, alguns terão exatamente duas faces pintadas e outros terão exatamente três faces pintadas.

Abaixo podemos ver um cubo de lado 4 cm ($N = 4$) após Ana pintá-lo e cortá-lo.



Ana contou quantas faces estavam pintadas em cada cubinho cortado do cubo acima e concluiu que, entre os 64 cubinhos, existem 8 cubinhos com nenhuma face pintada de preto, 24 cubinhos com exatamente uma face pintada, 24 cubinhos com exatamente duas faces pintadas e 8 cubinhos com exatamente três faces pintadas.

A sua tarefa é: dada a dimensão N do lado do cubo em centímetros, determine quantos cubinhos terão exatamente nenhuma, uma, duas e três faces pintadas de preto após Ana pintar e cortar o cubo.

Entrada

A entrada contém uma única linha com um único inteiro N , a dimensão do cubo em centímetros.

Saída

Seu programa deverá imprimir quatro linhas, cada uma contendo um único inteiro:

- A primeira linha deve conter o número de cubinhos com nenhuma face pintada de preto.
- A segunda linha deve conter o número de cubinhos com exatamente uma face pintada.
- A terceira linha deve conter o número de cubinhos com exatamente duas faces pintadas.
- A quarta e última linha deve conter o número de cubinhos com exatamente três faces pintadas.

Restrições

- $2 \leq N \leq 100$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (30 pontos):** $N = 5$.
- **Subtarefa 3 (70 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 4	Exemplo de saída 1 8 24 24 8
Exemplo de entrada 2 2	Exemplo de saída 2 0 0 0 8

Jogo do Poder

Nome do arquivo: `poder.c`, `poder.cpp`, `poder.java`, `poder.js` ou `poder.py`

Jonathan está empolgado com a nova sensação do momento: o *Jogo do Poder*. Este jogo é jogado em uma matriz de N linhas e M colunas, na qual cada célula possui um monstro. O monstro na linha i e coluna j possui poder $P_{i,j}$.

No início do jogo, Jonathan escolhe um dos $N \times M$ monstros para jogar. O monstro escolhido por Jonathan se torna o *herói* do jogo e começa o jogo com o poder indicado em sua célula. Jonathan pode mover o herói ortogonalmente (isto é, para cima, baixo, direita ou esquerda) na matriz enquanto o herói estiver vivo. O herói não pode sair da matriz, mas pode visitar a mesma célula múltiplas vezes.

Toda vez que o herói entra em uma célula com um monstro vivo, ocorre uma batalha entre o herói e o monstro da célula. O herói ganha a batalha se, e somente se, o seu poder for maior ou igual ao poder do monstro. Caso contrário, o herói morre e perde o jogo em *game over*. Toda vez que o herói ganha uma batalha, o monstro derrotado morre (ou seja, a célula não possui mais nenhum monstro) e, como recompensa, o poder do monstro é somado ao poder do herói (ou seja, se o herói matar o monstro da célula (i, j) , o poder do herói aumenta em $P_{i,j}$).

Jonathan percebeu que o jogo pode ser injusto: mesmo que ele jogue de maneira ótima, dependendo de sua escolha de herói, pode ser possível matar todos os monstros, apenas alguns ou até mesmo nenhum monstro.

Decidido a “platinar” o jogo, Jonathan precisa saber o poder máximo que cada herói consegue alcançar (ou seja, o poder máximo possível de ser atingido ao iniciar o jogo em cada célula da matriz) se o jogo for jogado de forma ótima. Felizmente, ele descobriu que os alunos da OBI (Organização dos Bons Informáticos) recentemente resolveram o *Jogo da Vida*, seu terceiro jogo favorito (atrás do *Jogo do Poder* e do *Jogo de Corrida*, claro), então ele pediu a sua ajuda novamente! Determine, para cada herói, o poder máximo que ele consegue alcançar caso Jonathan jogue de forma ótima.

Entrada

A primeira linha de entrada contém dois inteiros N e M , o número de linhas e o número de colunas da matriz, respectivamente.

As próximas N linhas contém M inteiros cada. O j -ésimo inteiro da i -ésima linha contém o poder $P_{i,j}$ do monstro na i -ésima linha e j -ésima coluna.

Saída

O seu programa deverá imprimir N linhas, cada uma contendo M inteiros. O j -ésimo inteiro da i -ésima linha deve ser o poder máximo que Jonathan consegue alcançar caso ele escolha como herói o monstro da célula (i, j) e jogue de maneira ótima.

Restrições

- $1 \leq N \leq 100\,000$
- $1 \leq M \leq 100\,000$
- $1 \leq N \times M \leq 100\,000$
- $1 \leq P_{i,j} \leq 1\,000\,000\,000$ para todo $1 \leq i \leq N$ e $1 \leq j \leq M$

Atenção: Observe que não é possível declarar $100\,000 \times 100\,000$ inteiros (com matriz, vetor etc.) sem estourar o limite de memória (isto causaria erros no programa pois tentaria usar dezenas de GB de memória). Preste atenção ao limite $N \times M \leq 100\,000$, que garante que a matriz sempre terá no máximo 100 000 células.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta sub tarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (11 pontos):**
 - $N = 1$
 - $M \leq 1000$
 - $1 \leq P_{1,j} \leq 2$ para todo $1 \leq j \leq M$
- **Subtarefa 3 (13 pontos):**
 - $N = 1$
 - $M \leq 1000$
- **Subtarefa 4 (29 pontos):**
 - $N = 1$
- **Subtarefa 5 (17 pontos):**
 - $N \leq 30$
 - $M \leq 30$
- **Subtarefa 6 (30 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 2 3 2 3 9 1 7 200	Exemplo de saída 1 6 6 22 1 22 222
Exemplo de entrada 2 1 7 6 3 10 1 20 7 7	Exemplo de saída 2 9 3 54 1 54 14 14

Exemplo de entrada 3	Exemplo de saída 3
5 6 10 10 10 10 10 10 10 10 1 1 1 10 10 10 10 1 10 10 10 10 10 4 10 10 10 10 10 10 10 2	250 250 250 250 250 250 250 250 8 8 8 250 250 250 250 8 250 250 250 250 250 8 250 250 250 250 250 250 250 2

Game Show

Nome do arquivo: `game.c`, `game.cpp`, `game.java`, `game.js` ou `game.py`

Populares no mundo inteiro, os *game shows* são programas de televisão ou internet nos quais os participantes competem em jogos para ganhar prêmios. A rede de televisão da sua cidade lançou um novo *game show* que funciona da seguinte forma:

- O jogo é jogado por um único participante e possui diversas salas numeradas a partir do número 1.
- Ao entrar em uma sala, o participante deve escolher seguir por uma das duas portas à sua frente: a porta da esquerda e a porta da direita. Na sala i , a porta da esquerda leva à sala $2 \cdot i$ enquanto que a porta da direita leva à sala $2 \cdot i + 1$. Por exemplo, na sala 1, a porta da esquerda leva à sala 2 e a porta da direita leva à sala 3. Da mesma forma, a porta da esquerda da sala 2 leva à sala $2 \cdot 2 = 4$ e a porta da direita da sala 4 leva à sala $2 \cdot 4 + 1 = 9$.
- No início do jogo, o participante é colocado na sala de número 1, em frente às portas para as salas 2 e 3, e recebe um papel com uma sequência de instruções indicando por qual porta ele deve seguir a cada momento do jogo. Cada instrução é representada por uma letra E ou D, indicando que ele deve seguir pela porta da esquerda ou da direita, respectivamente.
- As salas não possuem indicação de seus números e o desafio é não perder as contas: o participante ganha o jogo se ele conseguir chegar na sala final correta (ou seja, seguir pela porta correta a cada passo de acordo com as instruções) e, ao chegar nesta sala final, dizer corretamente o número dela.

Por exemplo, suponha que a sequência de instruções seja **EEDE**. Isso significa que o participante deve escolher as portas esquerda, esquerda, direita e esquerda, nesta ordem. Podemos ver no diagrama abaixo que, neste caso, o participante termina na sala de número 18. Portanto, ao chegar na última sala, o participante deve dizer o número 18 para vencer.

$$1 \xrightarrow{E} 2 \xrightarrow{E} 4 \xrightarrow{D} 9 \xrightarrow{E} 18$$

Você se inscreveu para participar do *game show*, mas percebeu que ele exige muita habilidade com cálculos matemáticos e decidiu praticar. Para isso, você quer escrever um programa que te ajude a conferir os cálculos. Dada uma sequência de instruções, determine qual o número da sala na qual o participante terminará o jogo se ele seguir corretamente as instruções.

Entrada

A primeira linha da entrada contém um inteiro N , o número de instruções dadas no início do jogo.

A segunda e última linha contém uma cadeia de N caracteres $a_1a_2 \dots a_N$, cada um deles sendo ‘E’ ou ‘D’, indicando as instruções dadas no início do jogo. No i -ésimo passo, se $a_i = \text{‘E’}$ o participante deve escolher a porta da esquerda e se $a_i = \text{‘D’}$ ele deve escolher a porta da direita.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o número da sala final caso o participante siga corretamente as instruções.

Restrições

- $1 \leq N \leq 20$
- $a_i = \text{‘E’}$ ou ‘D’ para $1 \leq i \leq N$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (30 pontos):** $N = 2$ (veja o exemplo 2).
- **Subtarefa 3 (30 pontos):** O participante deve ir para a esquerda em todos os passos, ou seja, $a_i = 'E'$ para todo $1 \leq i \leq N$ (veja o exemplo 3).
- **Subtarefa 4 (40 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 4 EEDE	Exemplo de saída 1 18
Exemplo de entrada 2 2 ED	Exemplo de saída 2 5
Exemplo de entrada 3 8 EEEEEEEE	Exemplo de saída 3 256

Salada de Frutas

Nome do arquivo: `frutas.c`, `frutas.cpp`, `frutas.java`, `frutas.js` ou `frutas.py`

Para a sobremesa do almoço de domingo, a mãe de Juninho vai preparar uma salada de frutas. Como ela é uma pessoa muito ocupada, pediu a Juninho que vá à feira para comprar as frutas. A mãe de Juninho deseja que a salada de frutas seja bem diversificada (da última vez, Juninho mencionou que tinha laranja demais). Por isso, ela deu a Juninho R reais e pediu para que ele volte da feira com o máximo de tipos de fruta diferentes que ele puder comprar.

Para simplificar a tarefa de Juninho, sua mãe lhe deu a tabela de preços da feira do bairro, onde existem diversas barracas. A tabela de preços é uma lista na qual cada item indica que uma determinada fruta está sendo vendida por um determinado preço na feira. Como a mesma fruta pode ser vendida em barracas diferentes, na tabela de preços é possível encontrar a mesma fruta várias vezes, cada vez com um preço diferente.

Juninho até poderia fazer as contas no papel para descobrir quantos tipos de fruta ele pode levar para casa, mas ele prefere ir à feira o quanto antes para voltar logo e assistir anime. Por isso, ele te contratou como programador.

Sua tarefa é: dada a tabela da feira contendo os tipos e preços das frutas, determine o número máximo de tipos de fruta que Juninho consegue comprar com os R reais que sua mãe lhe deu.

Entrada

A primeira linha da entrada contém dois inteiros R e N indicando, respectivamente, o valor em reais que a mãe de Juninho deu a ele e o número de frutas listadas na tabela da feira.

As próximas N linhas descrevem a tabela de preços. A i -ésima destas linhas contém dois inteiros T_i e P_i , o tipo e o preço em reais, respectivamente, da i -ésima fruta da tabela. O tipo de uma fruta é representado por um inteiro entre 1 e 100.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o número máximo de tipos de fruta diferentes que Juninho consegue comprar usando os R reais de sua mãe.

Restrições

- $1 \leq R \leq 10\,000$
- $1 \leq N \leq 100$
- $1 \leq T_i \leq 100$ para $1 \leq i \leq N$
- $1 \leq P_i \leq 100$ para $1 \leq i \leq N$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.

- **Subtarefa 2 (33 pontos):** $T_i = 1$ para todo $1 \leq i \leq N$, ou seja, existe apenas um tipo de fruta na tabela (*veja o exemplo 2*).
- **Subtarefa 3 (29 pontos):** $T_i \neq T_j$ para $i \neq j$, ou seja, todos os tipos de fruta na tabela são distintos (*veja o exemplo 3*).
- **Subtarefa 4 (38 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

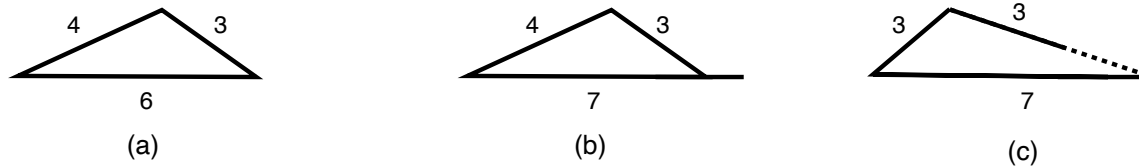
Exemplo de entrada 1 50 6 1 22 3 30 2 10 4 12 3 19 2 25	Exemplo de saída 1 3
Exemplo de entrada 2 10 4 1 18 1 11 1 31 1 14	Exemplo de saída 2 0
Exemplo de entrada 3 42 6 3 5 8 22 1 58 72 14 37 22 5 100	Exemplo de saída 3 3

Trio de Palitinhos

Nome do arquivo: `trio.c`, `trio.cpp`, `trio.java`, `trio.js` ou `trio.py`

Elisa está muito empolgada com o novo lançamento da OBI (Organização de Brincadeiras Infantis), o jogo *Trio de Palitinhos*, inspirado no clássico jogo dos palitos coloridos conhecido em algumas regiões como *Pega Varetas*.

No *Trio de Palitinhos*, existem N palitinhos retos de diversos tamanhos e numerados de 1 a N . O objetivo do jogo é selecionar três palitinhos e formar um triângulo com eles, de modo que cada palitinho represente exatamente um lado do triângulo. Não é permitido que algum lado possua um buraco, nem que um pedaço de algum palitinho “sobre” para fora do triângulo. A figura (a) abaixo ilustra um trio permitido e as figuras (b) e (c) ilustram trios proibidos de acordo com as regras do jogo.



Na aula de geometria, Elisa aprendeu sobre a *desigualdade triangular*, que diz que, em todo triângulo, a soma dos tamanhos de quaisquer dois lados é estritamente maior que o tamanho do terceiro lado. Enquanto jogava, Elisa percebeu que a recíproca também é verdadeira: dados três tamanhos satisfazendo essa condição, sempre é possível formar um triângulo com lados destes tamanhos. Isso explica porque é possível formar um triângulo com lados de tamanhos 3, 4 e 6, mas é impossível formar um triângulo com lados de tamanhos 3, 4 e 7 ($3 + 4$ não é estritamente maior que 7).

Agora, Elisa está curiosa para saber o quão difícil é ganhar o jogo, e por isso pediu a sua ajuda. Dados os tamanhos dos N palitinhos, determine quantos trios distintos de palitinhos existem com os quais é possível formar um triângulo (ou seja, que satisfazem a desigualdade triangular). Observe que a ordem de escolha dos três palitinhos não importa, mas palitinhos diferentes devem ser considerados diferentes mesmo que possuam o mesmo tamanho (*veja a explicação do exemplo 1*).

Entrada

A primeira linha da entrada contém um inteiro N indicando o número de palitinhos no jogo de Elisa.

A segunda linha de entrada contém N inteiros A_i separados por um espaço em branco, onde A_i é o tamanho em centímetros do i -ésimo palitinho.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o número de trios (não-ordenados) de palitinhos com os quais é possível formar um triângulo.

Restrições

- $3 \leq N \leq 1500$
- $1 \leq A_i \leq 1\,000\,000\,000$ para $1 \leq i \leq N$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (30 pontos):** $N \leq 100$.
- **Subtarefa 3 (24 pontos):** Existem exatamente dois tamanhos distintos de palitinhos (*veja o exemplo 3*). Formalmente,
 - $A_1 \neq A_N$,
 - $A_i = A_1$ ou $A_i = A_N$ para todo $1 \leq i \leq N$.
- **Subtarefa 4 (17 pontos):** $A_i \leq 100$ para todo $1 \leq i \leq N$.
- **Subtarefa 5 (29 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 5 7 4 3 6 3	Exemplo de saída 1 6
Exemplo de entrada 2 8 20 4 7 1 3 7 6 12	Exemplo de saída 2 12
Exemplo de entrada 3 6 9 5 5 5 9 5	Exemplo de saída 3 20

Remove Dígitos

Nome do arquivo: `remove.c`, `remove.cpp`, `remove.java`, `remove.js` ou `remove.py`

Beatriz continua se divertindo inventando jogos sobre dígitos. Sua mais nova invenção é o jogo *Remove Dígitos*.

Durante o jogo, Beatriz possui um número inteiro positivo N . A cada rodada, ela pode subtrair de N o valor de um dos dígitos presentes nele, reduzindo o valor de N . Por exemplo, se $N = 437$, Beatriz pode escolher uma das seguintes operações:

- subtrair 4, obtendo 433;
- subtrair 3, obtendo 434;
- subtrair 7, obtendo 430.

Observe que, após cada rodada, o número de Beatriz diminui. O objetivo do jogo é transformar o número em 0 o mais rápido possível.

O exemplo abaixo ilustra uma das maneiras de ganhar o jogo começando com $N = 23$, para o qual o mínimo de rodadas necessárias é 5. Observe que existe mais de uma sequência de operações que transforma 23 em 0 com cinco rodadas, mas é impossível transformar 23 em 0 com quatro ou menos rodadas.

$$23 \xrightarrow{-2} 21 \xrightarrow{-2} 19 \xrightarrow{-9} 10 \xrightarrow{-1} 9 \xrightarrow{-9} 0$$

Beatriz pediu sua ajuda para verificar os resultados do jogo: dado o número inicial N , determine o número mínimo necessário de rodadas de *Remove Dígitos* para transformar N em 0.

Entrada

A entrada é composta por uma única linha contendo um único inteiro, o número N com o qual Beatriz inicia o jogo.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o número mínimo de rodadas de *Remove Dígitos* necessárias para transformar o número N em 0.

Restrições

- $1 \leq N \leq 100\,000$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (18 pontos):** $N \leq 20$.
- **Subtarefa 3 (36 pontos):** $N \leq 100$.

- **Subtarefa 4 (46 pontos):** Sem restrições adicionais.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (elas não precisam ser resolvidas em ordem). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1 23	Exemplo de saída 1 5
Exemplo de entrada 2 14	Exemplo de saída 2 3
Exemplo de entrada 3 437	Exemplo de saída 3 75